

CiteBud

Schema Design & DDL

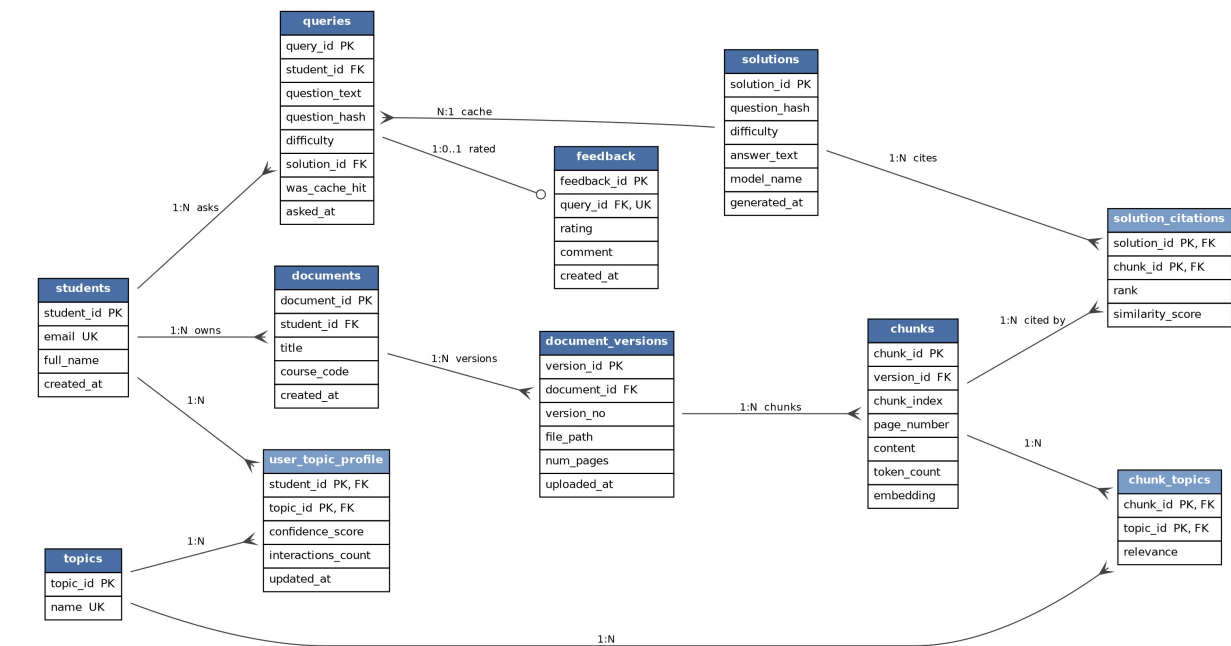
Project: CiteBud - Adaptive Study Assistant (RAG Pipeline) **DBMS:** PostgreSQL 14+ with `pgvector`

1. Design Philosophy

This is a **database-first** system — the RAG layer sits *on top of* a normalized relational core, not the other way around. Every feature of the system (versioning, topic modeling, caching, citations, feedback, user-topic profiles) is realized as a concrete schema construct rather than as application-layer bookkeeping. The schema has **11 tables**, and each one earns its place against a specific feature requirement.

#	Table	Feature it realizes
1	students	User identity
2	documents	Logical document identity (survives re-upload)
3	document_versions	Versioning system — immutable snapshots
4	chunks	Retrieval corpus + vector embeddings
5	topics	Topic extraction layer (controlled vocabulary)
6	chunk_topics	M:N tagging, with per-pair relevance
7	solutions	LLM answers, content-addressed for caching
8	queries	Question log; points at (possibly shared) solution
9	solution_citations	Many-to-many citation tracking with scores
10	feedback	Feedback loop — per-query ratings
11	user_topic_profile	User-adaptive learning — confidence per topic

2. ER Diagram



ER Diagram

Cardinalities Justified

Relationship	Cardinality	Why
students → documents	1 : N	Each upload belongs to exactly one student.
documents → document_versions	1 : N	Re-uploading creates a new version; old versions preserved for reproducibility.
document_versions → chunks	1 : N	Chunking happens per snapshot, so chunks are bound to a version, not a logical document.
chunks ↔ topics	M : N	A chunk can cover several topics; a topic spans many chunks. <code>relevance</code> is a property of the <i>pairing</i> , so it lives in the junction.
students → queries	1 : N	Full history kept.
queries → solutions	N : 1	Caching. Multiple queries with the same normalized hash + difficulty reuse one <code>solutions</code> row; <code>was_cache_hit</code> flags reuse for analytics.
solutions ↔ chunks	M : N	An answer cites several chunks; a chunk can ground many answers over time. <code>rank</code> and <code>similarity_score</code> are properties of the retrieval event, so they live in <code>solution_citations</code> .
queries → feedback	1 : 0..1	At most one rating per query, enforced by <code>UNIQUE(query_id)</code> .
students ↔ topics	M : N via <code>user_topic_profile</code>	Associative entity with its own attributes (confidence, counters).

3. 3NF Justification

A relation is in **3NF** iff for every non-trivial FD $X \rightarrow A$, either X is a superkey or A is a prime attribute — i.e. no partial dependencies and no transitive dependencies on the primary key.

Functional dependencies, table by table

students — PK `student_id`, alternate key `email`

```
student_id → email, full_name, created_at
email      → student_id
```

Both determinants are superkeys. Satisfies 3NF (also BCNF).

documents — PK `document_id`

```
document_id → student_id, title, course_code, created_at
```

No `students` attributes (like `email`) are copied here, so there's no transitive dependency `document_id → student_id → email`. Satisfies 3NF.

document_versions — PK `version_id`, alternate key `(document_id, version_no)`

```
version_id          → document_id, version_no, file_path, num_pages, uploaded_at
(document_id, version_no) → version_id
```

Both determinants are superkeys. `num_pages` is a property of *this specific upload*, not of the logical document, so it correctly sits here. Satisfies 3NF.

chunks — PK `chunk_id`, alternate key `(version_id, chunk_index)`

```
chunk_id          → version_id, chunk_index, page_number, content, token_count, embedding
(version_id, chunk_index) → chunk_id
```

`page_number` depends on `chunk_id`, not on `version_id` alone — so it belongs here, not in `document_versions`. No transitive dependency through `version_id`. Satisfies 3NF.

topics — PK `topic_id`, alternate key `name`. Trivially 3NF.

chunk_topics — PK `(chunk_id, topic_id)`

```
(chunk_id, topic_id) → relevance
```

`relevance` depends on the *whole* composite key — it's a property of the pairing, not of the chunk or topic alone. No partial dependency. Satisfies 3NF.

solutions — PK `solution_id`, alternate key `(question_hash, difficulty)`

```
solution_id → question_hash, difficulty, answer_text, model_name, generated_at
(question_hash, difficulty) → solution_id
```

Both determinants are superkeys. Crucially, `student_id` is not stored here — a cached solution is shared across students, so attaching it to a specific student would be factually wrong *and* would create a transitive dependency. Satisfies 3NF.

queries — PK `query_id`

```
query_id → student_id, question_text, question_hash, difficulty, solution_id, was_cache_hit, asked_at
```

Note what is *not* here: `answer_text`, `model_name`, `generated_at`. Those depend on `solution_id`, not on `query_id`, so storing them would be the classic transitive dependency `query_id → solution_id → answer_text` and would break 3NF. We follow the FK instead. Satisfies 3NF.

solution_citations — PK `(solution_id, chunk_id)`, alternate key `(solution_id, rank)`

```
(solution_id, chunk_id) → rank, similarity_score
(solution_id, rank) → chunk_id
```

Every non-prime attribute depends on a full candidate key. `similarity_score` is a property of *this specific retrieval event*, not of the chunk in isolation — so it correctly lives in the junction. Satisfies 3NF.

feedback — PK `feedback_id`, alternate key `query_id`

```
feedback_id → query_id, rating, comment, created_at
query_id → feedback_id
```

Student identity and question text are *not* duplicated here — they're reachable through `query_id`. Storing them would create a transitive dependency. Satisfies 3NF.

user_topic_profile — PK `(student_id, topic_id)`

```
(student_id, topic_id) → confidence_score, interactions_count, updated_at
```

All non-prime attributes depend on the whole composite key. Confidence is inherently a property of the *pair*. Satisfies 3NF.

Decomposition Notes

Three relationships forced explicit decomposition:

1. **chunks ↔ topics (M:N)**. Storing topics as a comma-separated column on `chunks` would violate 1NF; attaching per-pair relevance to it would then cascade into partial-dependency violations. `chunk_topics` resolves both issues.
2. **solutions ↔ chunks (M:N)**. Same argument. Additionally, `similarity_score` and `rank` are event-scoped, not chunk-scoped, so the junction is the only correct home.
3. **queries ↔ solutions (many-to-one via caching)**. A naive design would inline the answer into `queries`, causing `answer_text` to be duplicated across every cache-hit query and creating a transitive dependency `query_id → (question_hash, difficulty) → answer_text`. Separating `solutions` out eliminates this and *is* the cache.

4. Constraint Summary

Constraint	Where	Purpose
PRIMARY KEY	every table	Row identity.
FOREIGN KEY	all non-root tables	Referential integrity.
UNIQUE	<code>students.email</code> ; <code>documents(student_id, title)</code> ; <code>document_versions(document_id, version_no)</code> ; <code>chunks(version_id, chunk_index)</code> ; <code>topics.name</code> ; <code>solutions(question_hash, difficulty)</code> ; <code>solution_citations(solution_id, rank)</code> ; <code>feedback.query_id</code>	Alternate keys & business rules.
CHECK	<code>num_pages > 0</code> ; <code>page_number ≥ 1</code> ; <code>chunk_index ≥ 0</code> ; <code>version_no ≥ 1</code> ; <code>token_count > 0</code> ; <code>difficulty ∈ {foundational, intermediate, advanced}</code> ; <code>similarity_score ∈ [-1, 1]</code> ; <code>relevance ∈ [0, 1]</code> ; <code>confidence_score ∈ [0, 1]</code> ; <code>rating ∈ [1, 5]</code> ; <code>rank ≥ 1</code> ; <code>interactions_count ≥ 0</code>	Domain validity.
ON DELETE CASCADE	<code>student → documents → versions → chunks</code> ; <code>student → queries → feedback</code> ; <code>student → user_topic_profile</code>	Prevents orphan rows; GDPR-friendly.
ON DELETE RESTRICT	<code>solution_citations → chunks</code>	Don't silently lose citation history if a chunk is re-indexed.
ON DELETE SET NULL	<code>queries.solution_id</code>	A purged solution leaves the query log intact.

5. Indexes (prep for Milestone 3)

Three indexes are created up front:

- `chunks_embedding_cos_idx` — `ivfflat` over `embedding` with `vector_cosine_ops`. Semantic half of hybrid retrieval.
- `chunks_version_idx` — B-Tree on `chunks(version_id)`. Used whenever retrieval is scoped to a specific document version; provides execution-plan evidence of B-Tree index use.
- `queries_hash_diff_idx` — B-Tree on `queries(question_hash, difficulty)`. Powers cache-hit analytics.

More indexes with before/after `EXPLAIN ANALYZE` evidence will be added later.

6. How to Run

```
createdb studydb
psql -d studydb -c "CREATE EXTENSION IF NOT EXISTS vector;"
psql -d studydb -f schema.sql
```

The script is idempotent for development: it drops existing tables in FK-safe order before recreating them. No manual edits required.

7. Mapping Features → Schema (Viva Cheat Sheet)

Feature	Where it lives
Version-controlled document collections	<code>documents</code> + <code>document_versions</code> , chunks bound to version
Hybrid retrieval (semantic + keyword)	<code>chunks.embedding</code> (pgvector) + future <code>tsvector</code> column
Query caching	<code>solutions(question_hash, difficulty)</code> UNIQUE; <code>queries.solution_id</code> reuse
Topic extraction layer	<code>topics</code> + <code>chunk_topics(relevance)</code>
M:N citation tracking with scores	<code>solution_citations(rank, similarity_score)</code>
Grounded generation (per-difficulty)	<code>solutions.difficulty</code> CHECK + citations FK
Feedback loop	<code>feedback(rating, comment)</code>
User-topic confidence profile	<code>user_topic_profile</code>
Reproducibility over time	Version-bound chunks + immutable solutions